# IMSE785: Project 1

Timothy Bailey
IMSE 785: Big Data Analytics
Kansas State University

## Introduction

### *Project Description*

This project focuses on processing and analyzing a large text dataset using the Hadoop MapReduce framework. The dataset used is Shakespeare's text file, which contains a large volume of unstructured textual data. The goal is to efficiently process this dataset in a distributed computing environment and extract insights from raw text.

Using Hadoop, a WordCount MapReduce job was executed to compute the frequency of each word in the dataset. However, the initial output contains unprocessed text, including inconsistencies such as uppercase and lowercase word variations, punctuation artifacts, and common stop words (e.g.,"the", "and", "of"). These issues could effect the quality of the results in the analysis.

### *Project Goals*

The objective of the project is to implement a Hadoop MapReduce job to process a large text dataset and generate frequency counts from Shakespeare's text. This involves using the Hadoop ecosystem to efficiently store, manage, and process large-scale unstructured text data in a distributed environment. After processing the data, the results are transferred from HDFS to a local machine for further analysis. Once the data is available locally, Python is used to perform data wrangling tasks, including converting text to lowercase. Also it is used for removing punctuation, non-alphabetic characters, and common stop words. Finally, the cleaned dataset is used to create word-frequency data visualizations and to analyze the most common words in the Shakespeare dataset.

## Inputs and Outputs

### *Input*

The input is a text file containing Shakespeare's complete works. The file was uploaded into the Hadoop Distributed File System (HDFS) and stored within the input directory used by the program. The text file served as the dataset for the word count analysis.

Before executing the MapReduce job, the input file was verified within HDFS using the following command:

Listing 1: Verifying Input File in HDFS

```
hdfs dfs -ls /user/tcbailey/input_files
```

The WordCount job processes the text by reading the file line by line. During the Map phase, each line is broken into individual words. Then HDFS generates an intermediate key-value pair for every word encountered. Each word is assigned a value of one, indicating a single occurrence. These intermediate records are then passed to the Shuffle and Sort phase, where Hadoop groups words together before sending them to the reducers for aggregation.

***Output***

The output of the WordCount MapReduce job is a frequency table containing every unique word found within the Shakespeare text and the total number of times that word appears. During the Reduce phase, Hadoop combines all intermediate values associated with the same word and calculates a final count.

The MapReduce job is executed using the following command:

Listing 2: Executing MapReduce in HDFS

```
hadoop jar ~/Project1/hadoop-examples.jar wordcount \
> -D yarn.app.mapreduce.am.staging-dir=/user/tcbailey/tmp \
> /user/tcbailey/input_files \
> /user/tcbailey/output
```

Once the MapReduce job is complete the output is written into an HDFS directory and viewed using:

Listing 3: Viewing Results

```
hdfs dfs -cat /user/tcbailey/output/part-r-00000 | head
```

The output file contained over 73,000 unique words and their respective frequency counts. This file served as the foundation for the remainder of the analysis. The output was then moved locally and imported into Python to complete the final portion of the project.

# Data Wrangling

The first step in the process was importing the Hadoop output into Python. The output file was loaded into a DataFrame containing two columns, word and count. To be consistent, words were converted to lowercase. Also, punctuation and special characters were removed. This prevented variations of the same word from being treated as separate entries.

Common articles, pronouns, and other non-informative words were removed from the text using the Natural Language Toolkit (NLTK) stopword library. The stopword list contains frequently occurring words such as, a, an, the, and, and but, which generally do not contribute meaningful information to text analysis. By filtering these words from the dataset, the analysis could focus on terms that better represented the content and themes of Shakespeare's works. The use of NLTK automated the stopword removal process and reduced the amount of manual filtering required.

Following the removal of all stopwords, the top 50 high-frequency words were reviewed and categorized according to their part of speech. Due to difficulties encountered with NLTK packages, the most frequent words were manually categorized. Only common nouns were retained, while pronouns, verbs, adjectives, and proper nouns were excluded. For example, words such as lord, king, queen, and heart were retained, while words such as thou, shall, and good were removed.

Following filtering to only nouns, the dataset was sorted in descending order based on frequency count. The ten most frequently occurring nouns were selected and stored in a new DataFrame for visualization. Table 1 presents the final set of nouns and their associated frequency counts.

Table 1: Top 10 Most Frequent Nouns in Shakespeare's Works

| Rank | Noun | Count | Cumulative % |
|---|---|---|---|
| 1 | lord | 3355 | 17.59 |
| 2 | king | 3317 | 34.99 |
| 3 | sir | 3019 | 50.82 |
| 4 | love | 2197 | 62.35 |
| 5 | man | 1874 | 72.17 |
| 6 | duke | 1167 | 78.29 |
| 7 | time | 1104 | 84.08 |
| 8 | heart | 1050 | 89.59 |
| 9 | queen | 1002 | 94.84 |
| 10 | lady | 983 | 100.00 |

To better visualize the distribution of nouns, a Pareto Chart was created using the final dataset. The Figure below illustrates the frequency of each noun in descending order. The chart shows that the nouns lord, king, and sir occur substantially more often than the other nouns in the top ten.

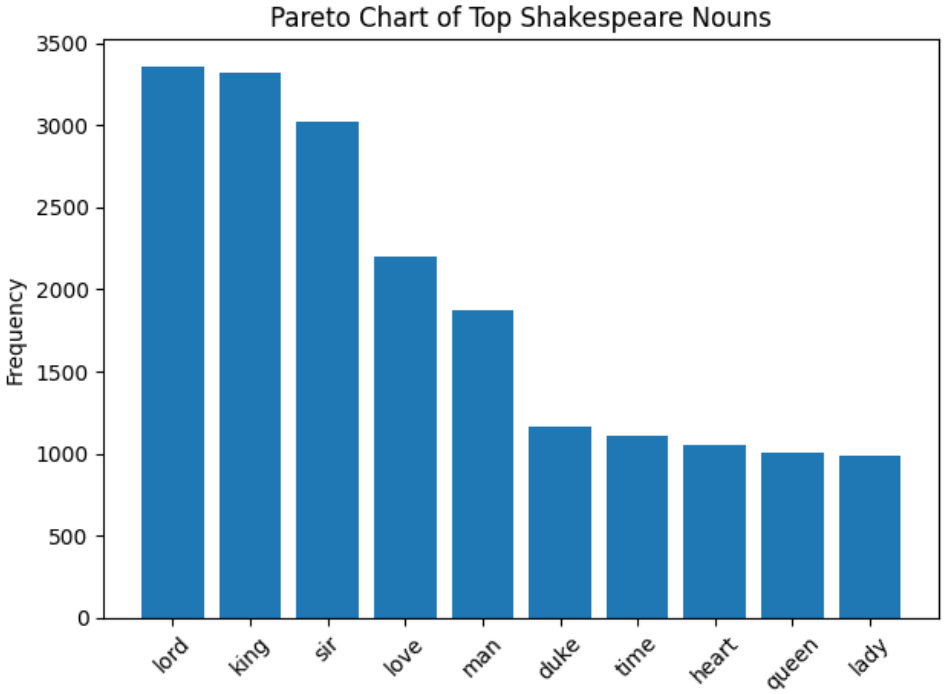


Figure 1: Top Shakespeare Nouns.

The final table and chart summarize the most frequent nouns identified in the Shakespeare text and demonstrate how data wrangling techniques can transform raw Hadoop output into meaningful analytical results.

## Systems Used

This project was completed using a Hadoop-based big data processing environment running on a Linux virtual machine. The Shakespeare text was stored within the HDFS and processed using the Hadoop MapReduce WordCount job to generate word frequency counts. The resulting output was imported into Python and analyzed using the Pandas and NLTK libraries within Visual Studio Code. Pandas was used for data manipulation and sorting, while NLTK was used to remove common stopwords. The final results were visualized using Matplotlib to create a sorted frequency histogram (Pareto Chart) of the most frequently occurring nouns.

# Appendix

### *Hadoop Commands*

The following commands summarizes the Hadoop MapReduce workflow used in this project. This includes verifying input files, executing the WordCount job, and retrieving the output for further analysis in Python.

Listing 4: Hadoop MapReduce Workflow (Cleaned Terminal Commands)

```

ls -l /user/tcbailey/Project1/

hadoop jar ~/Project1/hadoop-examples.jar wordcount \
    -D yarn.app.mapreduce.am.staging-dir=/user/tcbailey/tmp \
    /user/tcbailey/input_files \
    /user/tcbailey/output

hdfs dfs -ls /user/tcbailey/output

hdfs dfs -cat /user/tcbailey/output/part-r-00000 | head

hdfs dfs -get /user/tcbailey/output/part-r-00000 .
```

### *Python Code*

The following Python script was used to import the Hadoop WordCount output, clean the dataset, remove stopwords, identify the most frequent nouns, and generate the final Pareto chart.

Listing 5: Python Data Processing and Visualization Code

```
import pandas as pd
import matplotlib.pyplot as plt
import nltk
from nltk.corpus import stopwords

# Load dataset
shakespeare_df = pd.read_csv(
    "/Users/tcbailey/Documents/imse785/projects/project1/outputs/
       shakespeare_wordcount.txt",
    sep=r"\s+",
    header=None,
    names=["word", "count"]
)

# Clean text
shakespeare_df["word"] = shakespeare_df["word"].str.lower()
shakespeare_df["word"] = shakespeare_df["word"].str.replace(r"[^a
   -z']", "", regex=True)

# Remove stopwords
nltk.download("stopwords")
stop_words = set(stopwords.words("english"))
```

```
21
22 shakespeare_df = shakespeare_df[
23     ~shakespeare_df["word"].isin(stop_words)
24 ]
25
26 # Aggregating
27 shakespeare_df_clean = (
28     shakespeare_df.groupby("word", as_index=False)["count"]
29     .sum()
30     .sort_values("count", ascending=False)
31 )
32
33 # Filtering for top 10 nouns
34 top_nouns_df = pd.DataFrame({
35     "word": ["lord","king","sir","love","man",
36              "duke","time","heart","queen","lady"],
37     "count": [3355,3317,3019,2197,1874,
38               1167,1104,1050,1002,983]
39 })
40
41 # Sort values
42 top_nouns_df = top_nouns_df.sort_values(by="count", ascending=
       False)
43
44 # Plot Pareto chart
45 fig, ax = plt.subplots()
46 ax.bar(top_nouns_df["word"], top_nouns_df["count"])
47 ax.set_ylabel("Frequency")
48 plt.title("Pareto Chart of Top Shakespeare Nouns")
49 plt.xticks(rotation=45)
50 plt.tight_layout()
51 plt.show()
```